

LOOPS, COMMAND LINE ARGS & FILE READING! (LECTURE)

QUICK INVENTORY

- Lectures:
 - Monday's included **interactivity practice** and a tiny bit of sequences
 - Today's will have more **sequences** practice, some **for looping** practice, and some "expansion" material about file reading.
- Videos:
 - Monday's were about **sequences**, which we've touched just a bit
 - Today's were about **for loops**, which we (naturally) have not practiced yet

➡ Sorry that the videos and lectures are a little tangled this week; thanks for being diligent about watching and coming to class. More "on focus" after this week.

REMINDERS

- There is another check-in due before next lecture as always.
- TA OH M-F, Harry OH M 10:30-11:30am & W 2:45-3:45pm
- HW01 is due tonight (2/4) at 11:59pm
- Expect HW02 to be released early afternoon tomorrow
 - After today, you'll have just enough to get started, make a plan, and do the first task.
 - If you want to get a big head start, you can review the **Files** section from these slides. (We will probably not get through most of it today.)

INDEXING AND SLICING IN SEQUENCES

RECAP: INDEXING IN SEQUENCES

Sequences in Python are **indexable**: we can refer to values at specific positions in the sequence by their positions.

- first value lives at index `0`
- second value lives at index `1`

```
"indexing"  
01234567
```

Notice that "indexing" is a string with eight characters: since we start counting at `0`, the index of the last character is `7`.

RECAP: ACCESSING INDICES

To access ("read") the element at a particular position of a sequence, we use its index with the `[]` operator.

```
my_string = "colour"
```

```
print(my_string[0])
```

```
print(my_string[4])
```

```
#   'c'
```

```
#   'u'
```

RECAP: ACCESSING INDICES

To access ("read") the element at a particular position of a sequence, we use its index with the `[]` operator.

```
my_list = [8, 59, 22]
```

```
print(my_list[1])
```

```
#   59
```

```
print(my_list[1 + 1])
```

```
#   22 (i.e. my_list[2])
```

```
print(my_list[1] + 1)
```

```
#   60 (i.e. 59 + 1)
```

We can put any *integer expression* inside of the `[]` operator; its value is used as the index.

RECAP: ACCESSING INDICES

To access ("read") the element at a particular position of a sequence, we use its index with the `[]` operator.

```
my_tuple = ("five", "thousand", "acts", "of", "congress")

print(my_tuple[4])           # 🖨️ ➡️ 'congress'
print(my_tuple[-1])          # 🖨️ ➡️ 'congress'
print(my_tuple[len(my_tuple) - 1]) # 🖨️ ➡️ 'congress'
print(my_tuple[len(my_tuple)])  # 💀 IndexError: tuple index out of
```

We can access the last element of the sequence by using the index *one less than the length of the sequence*, or with the shortcut of `-1`.



A **train** is like a sequence of train cars. Train cars typically have names like `"R234B"` or `"Z938A"`. We can model a **train** as a list of strings:

- the first element is the name of the lead car; the last element is the name of the rear car.
- cars that are "old" start with the letter `Z`; newer cars start with any other letter.
- a train might have any number of cars..

Let's help solve some train problems!

IDENTIFY THE ENDS (THINK, CONFER, DISCUSS)

Here are two examples of what a train-as-list might look like.

```
a_train = ["D418X", "G042K", "Z944X", "N424L", "T861N", "N755H"]  
b_train = ["Z606L", "F320R", "M498D", "Z760K", "A887M", "P156V", "I057G"]
```

Given a variable storing some (unknown) train, how would we complete the following snippet to print out the name of the lead (first) car?

```
my_train = ... # (some list like above)  
first_car_name = _____ # 🤔 Fill in this blank! 🤔  
print(f"First car in this train is {first_car_name}")
```

IDENTIFY THE ENDS (THINK, CONFER, DISCUSS)

Here are two examples of what a train-as-list might look like.

```
a_train = ["D418X", "G042K", "Z944X", "N424L", "T861N", "N755H"]  
b_train = ["Z606L", "F320R", "M498D", "Z760K", "A887M", "P156V", "I057G"]
```

Given a variable storing some (unknown) train, how would we complete the following snippet to print out the name of the last car?

```
my_train = ... # (some list like above)  
last_car_name = _____ # 🤔 Fill in this blank! 🤔  
print(f"Last car in this train is {last_car_name}")
```

TRAIN DATING (THINK, CONFER, DISCUSS)

Remember that **old cars** have names that start with the letter **Z**

```
new_name = "D418X"  
old_name = "Z606L"
```

Given a variable storing some (unknown) train name, how would we complete the following snippet to print out a warning if a car is old?

```
my_name = ...  
is_old = _____ # 🤔 Fill in this blank! 🤔  
if is_old:  
    print(f"Beware! {my_name} is an old car.")
```

SAFETY INSPECTION C12

Remember that **old cars** have names that start with the letter **Z**. A train is **dangerous** if its first or last cars are old. Fill in this program to print a message only when the train is dangerous. (The last blank is the only "new" one.)

```
my_train = ...
first_car_name = _____
last_car_name = _____
first_is_old = _____
last_is_old = _____
if _____:
    print("DANGER! This train is not up to safety standards.")
else:
    print("Enjoy your ride!")
```

COMMAND LINE ARGUMENTS

COMMAND LINE ARGS

When we run a program we usually type something like

```
python my_program.py
```

We can also specify some information when we run the program through
Command Line Arguments:

```
python greeting.py Harry
```

The stuff written after `python greeting.py` is available to the program immediately when we start it!

SYS.ARGV

We can `import sys` and use `sys.argv` to get command line args

Consider the file `args.py`

```
import sys
print(sys.argv)
```

Run with:

```
python args.py Ta lie Ma ssa chi
```

prints: `['args.py', 'Ta', 'lie', 'Ma', 'ssa', 'chi']`

Note: `argv` has EVERYTHING after `python` in the command. Also note that it is a

COMMAND LINE ARGS

Consider we have the following program called `greeting_argv.py`.

```
import sys

print(f"Hello, {sys.argv[1]}!")
```

If we run it with the command: `python greeting_argv.py Harry` then:

- `sys.argv` is the list `["greeting_argv.py", "Harry"]`
- `sys.argv[1]` is `"Harry"`

And so the printed output is: `Hello, Harry!`

COMMAND LINE ARGS

Consider we have the following program called `greeting_argv.py`.

```
import sys
print(f"Hello, {sys.argv[1]}!")
```

What if we just did `python greeting_argv.py`?

```
Traceback (most recent call last):
  File "/some_path/greeting_argv.py", line 3, in <module>
    print("Hello, " + sys.argv[1] + "!")
IndexError: list index out of range
```

💀 No CLA at position `1` 💀



FOR LOOPS

FOR LOOPS

```
for character in "Hello!":  
    print(character)
```

`for` "goes over" each item in a sequence.

```
'H'  
'e'  
'l'  
'l'  
'o'  
'!'
```

The loop will:

- Run's the code in the loop once for each item in the sequence
- Each time we run the "body" (code *in* the loop), our loop variable (`character` in this example) will store the next element from the sequence.

REVIEW: FOR LOOPS

This can be used to repeat code! Instead of:

```
print("Hello!!!!")
print("Hello!!!!")
...
print("Hello!!!!")
```

We can do something better:

```
for i in range(0, 10): # loop 10 times (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
    print("Hello!!!!")
    # note how we aren't required to use i
```

FOR LOOP PRACTICE (DON'T JUST RUN IT)

Consider the following loop, what is the final value of `nums`? (S7)

```
nums = []  
for i in range(0, 4):  
    nums.append(i * len(nums))
```

What is the value of `skills` after this code is run? (S8)

```
skills = [18, 88, 20, 82, 91, 78, 15]  
for i in range(len(skills) - 1):  
    if skills[i] >= 20:  
        skills[i] = skills[i] + 7  
    else:  
        skills[i] += 2
```

WHAT DOES THIS DO? (THINK, CONFER, DISCUSS)

(and how does it work?)

```
strings = ["My", "Anti", "Aircraft", "Friend"]

longest_str = strings[0]

for string in strings:
    if len(string) > len(longest_str):
        longest_str = string

print(f"The longest string is {longest_str}.")
```

WHAT DOES THIS DO? (THINK, CONFER, DISCUSS)

```
strings = ["My", "Anti", "Aircraft", "Friend"]
# To start, guess that the first string is the longest.
# Might be right or wrong, but we need to start somewhere.
longest_str = strings[0]

# Visit each string in the list one by one...
for string in strings:
    # if the current string is longer than the biggest we've seen...
    if len(string) > len(longest_str):
        # ...write down the current string as the new biggest so far.
        longest_str = string
    # if the current string is not bigger, do nothing & keep looping
# after we look at all strings, the longest "so far" is THE longest.
print(f"The longest string is {longest_str}.")
```


REVIEW: FOR LOOPS W/ ENUMERATE ()

We can use `enumerate()` to get the indices and items of the sequence paired together:

```
nums = [3, 2, 5]
for index, item in enumerate(nums):
    print(f"Index {index}: {item}")
```

```
Index 0: 3
Index 1: 2
Index 2: 5
```

Each iteration of the loop gives us two values to work with, so we choose two variable names.

ENUMERATION PRACTICE

We want to write some code to find *the index of the longest string* in a list. Fill in the loop body. **C14**

```
strings = ["My", "Anti", "Aircraft", "Friend"]

index = 0
longest_str = strings[0]

for i, string in enumerate(strings):
    # TODO: Fill out this loop

print(f"The longest string is {longest_str} at index {index}")
```

FILES

FILES

On computers we have things called **files**.

Files are where we store information that the computer can still access even after the computer turns off and on again.

We have already use files before: our programs are stored in `.py` files.

For now, we can assume that the contents of files are all characters. For text files like these, we think of files as being made of a "sequence" of lines of text.

```
Is there anybody      # first line
                      # second line
out there?           # third line
```

OPEN() AND CLOSE()

To read a file, we need to create a file "object" associated with that file. We can create a variable holding a file object with the `open()` call.

```
# opens the file "filename.txt" with "r" (Reading) enabled  
example_file = open("filename.txt", "r")
```

When we are completely done with a file, we need to close it

```
example_file.close()
```

...but what do we do in between the opening and closing?

READLINE()

Once we have an open file object, we can use `readline()` to read a line from the file. TRY IT! `python first_three_lines.py hello.txt`

```
# on website as first_three_lines.py
import sys
my_file = open(sys.argv[1], "r")
for i in range(3):
    line = my_file.readline()
    print(line)
my_file.close()
```

The next time we call `readline()` we get the next line of the file. These File objects remembers our position in the file.

STRIP()

We can use the `.strip()` function on a string to remove any leading or trailing white space.

Whitespace characters are characters that just add "spacing" but don't display like typical characters: tab (`'\t'`), space (`' '`), newline (`'\n'`)

`readline()` returns a line from a file, with the newline character (`'\n'`) at the end. We can remove this newline if we call `strip()`

```
line = my_file.readline().strip()
```

SPLIT()

What if we want to get all the "words" that make-up a string?

The `split` function returns a list of strings containing all the words that have whitespace between them.

```
line = "I am 2 late"  
tokens = line.split()  
print(line) # ["I", "am", "2", "late"]
```

Note how all the elements are still strings!

PRACTICE:

Assume we have a file named `beep.boop` with the layout:

```
this file has 3 lines after this  
line 0  
line 1  
line 2
```

Write some code that can read a file like this and print out all the lines but the first.

Assume that the file can have any number instead of `3` (C16)

- You should probably use: `open(filename, "r")`, `file.readline()`, `file.close()`, `string.strip()`, `string.split()`

ENUMERATION PRACTICE ANSWERS

```
strings = ["My", "Anti", "Aircraft", "Friend"]

index = 0
longest_str = strings[0]

for i, string in enumerate(strings):
    if len(string) > len(longest_str):
        index = i
        longest_str = string

print(f"The longest string is {longest_str} at index {index}")
```